Technische Hochschule Nürnberg

Fakultät Informatik

RaspberryPi CyberSec Lab: Development of a Penetration Testing Platform

Bachelorarbeit im Studiengang Elektro- und Informationstechnik

vorgelegt von

Jonas Schmitt

Matrikelnummer 3619551

Erstgutachter: Prof. Dr. Aykan D. Inan

Zweitgutachter: Jörg Arndt

© 2025

Dieses Werk einschließlich seiner Teile ist **urheberrechtlich geschützt**. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtgesetzes ist ohne Zustimmung des Autors unzulässig und strafbar. Das gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmungen sowie die Einspeicherung und Verarbeitung in elektronischen Systemen.

Hinweis: Diese Erklärung ist mit Originalunterschrift (nicht gescannt) in das Papierexemplar der Abschlussarbeit fest einzubinden. Eine Spiralbindung ist nicht zulässig. Das digitale Exemplar enthält einen Scan der Erklärung mit Unterschrift.

Prüfungsrechtliche Erklärung der/des Studierenden

Angaben des bzw. der Studierenden:

Name: Vorname: Matrikel-Nr.:

Fakultät: Studiengang:

Semester:

Titel der Abschlussarbeit:

Ich versichere, dass ich die Arbeit selbständig verfasst, nicht anderweitig für Prüfungszwecke vorgelegt, alle benutzten Quellen und Hilfsmittel angegeben sowie wörtliche und sinngemäße Zitate als solche gekennzeichnet habe. Außerdem versichere ich, dass der Hauptteil des Papierexemplares und des digitalen Exemplares identisch sind. Das digitale Exemplar enthält, falls gefordert, lediglich weitere Anlagen.

Ort, Datum, Unterschrift Studierende/Studierender

Erklärung der/des Studierenden zur Veröffentlichung der vorstehend bezeichneten Abschlussarbeit

Die Entscheidung über die vollständige oder auszugsweise Veröffentlichung der Abschlussarbeit liegt grundsätzlich erst einmal allein in der Zuständigkeit der/des studentischen Verfasserin/Verfassers. Nach dem Urheberrechtsgesetz (UrhG) erwirbt die Verfasserin/der Verfasser einer Abschlussarbeit mit Anfertigung ihrer/seiner Arbeit das alleinige Urheberrecht und grundsätzlich auch die hieraus resultierenden Nutzungsrechte wie z.B. Erstveröffentlichung (§ 12 UrhG), Verbreitung (§ 17 UrhG), Vervielfältigung (§ 16 UrhG), Online-Nutzung usw., also alle Rechte, die die nicht-kommerzielle oder kommerzielle Verwertung betreffen.

Die Hochschule und deren Beschäftigte werden Abschlussarbeiten oder Teile davon nicht ohne Zustimmung der/des studentischen Verfasserin/Verfassers veröffentlichen, insbesondere nicht öffentlich zugänglich in die Bibliothek der Hochschule einstellen.

Hiermit ☐ genehmige ich, wenn und soweit keine entgegenstehenden Vereinbarungen mit Dritten getroffen worden sind,

☐ genehmige ich **nicht,**

dass die oben genannte Abschlussarbeit durch die Technische Hochschule Nürnberg Georg Simon Ohm, ggf. nach Ablauf einer mittels eines auf der Abschlussarbeit aufgebrachten Sperrvermerks kenntlich gemachten Sperrfrist

von Jahren (0 - 5 Jahren ab Datum der Abgabe der Arbeit),

der Öffentlichkeit zugänglich gemacht wird. Im Falle der Genehmigung erfolgt diese unwiderruflich; hierzu wird der Abschlussarbeit ein Exemplar im digitalisierten PDF-Format an die Betreuer übermittelt. Bestimmungen der jeweils geltenden Studien- und Prüfungsordnung über Art und Umfang der im Rahmen der Arbeit abzugebenden Exemplare und Materialien werden hierdurch nicht berührt.

Ort, Datum, Unterschrift Studierende/Studierender

Datenschutz: Die Antragstellung ist regelmäßig mit der Speicherung und Verarbeitung der von Ihnen mitgeteilten Daten durch die Technische Hochschule Nürnberg Georg Simon Ohm verbunden. Weitere Informationen zum Umgang der Technischen Hochschule Nürnberg mit Ihren personenbezogenen Daten sind unter nachfolgendem Link abrufbar: https://www.th-nuernberg.de/datenschutz/

Kurzdarstellung

Diese Arbeit befasst sich mit der Entwicklung einer Testplatform für Cyberangriffe mit dem Fokus auf Funkverbindungen. Im Rahmen den Projekts wurde auf Basis von RaspberryPi OS Software entwickelt, um Umgebungen zu schaffen, in denen verschiedene Cyberangriffe getestet werden können.

Abstract

Contents

1. Introduction **1**
1.1. This is an Important Section 1
1.1.1. And an even more important subsection 1

2. Wifi Basics **3**
2.1. WiFi History 3
2.2. MAC layer 3
2.2.1. Architecture 3
2.2.2. MAC Frames 4
2.2.3. Medium Access 6
2.2.4. Connection Establishment and Termination 6
2.3. PHY layer 7
2.4. Security 8
2.4.1. Encryption 8
2.4.2. Authentication 9

3. Architecture **11**
3.1. Hardware 11
3.2. Software 12

4. Worklog **17**

5. Wifi Attacks **21**
5.1. Overview 21
5.2. War Driving 22
5.3. Rogue Access Point and Evil Twin 22
5.4. Cracking WEP 22

6. Penetration Testing **25**
6.1. Abc 25

7. Educational Use **27**
7.1. abc 27

8. Outlook **29**

9. Summary 31

A. Supplemental Information 33

List of Figures 37

List of Tables 39

List of Listings 41

Bibliography 43

Glossary 45

Chapter 1.

Introduction

"Dennoch geht man davon aus, dass Wirtschaftsspionage allein in Deutschland Schäden von von vielen Milliarden Euro jährlich verursacht." [Hell 23, page 3] "Eine funktionierende IT-Infrastruktur ist für ein Unternehmen unerlässlich, doch wird deren Bedeutung of unterschätzt. Wie verschiedene Untersuchungen zeigen, überlebt ein Unternehmen bei einem anhaltenden Totalausfall der IT sehr wahrscheinlich nur wenige Tage." [Hell 23, page 3]

1.1. This is an Important Section

It is possible to reference glossary entries as library as an example.

1.1.1. And an even more important subsection

Chapter 2.

Wifi Basics

2.1. WiFi History

In 1985 the american Federal Communications Commission ruled serveral bands in the 2.4 GHz radio band to be unlicensed, meaning everyone can broadcast radiosignals in these frequencies. At the beginning of the 90's, this lead to the development of different wireless communication protocols by multiple companies. To ensure interoperability between these protocols, the IEEE 802.11 standard was created and released, and WLAN was born. In 1999 the Wi-Fi Alliance (WFA) was founded to further improve compatibility, nowadays consisting of over 750 international companies. [Sank 21, page 1-3], [Wiki 25]

Since its inception over 25 years ago, Wi-Fi has gone through constant development, leading to six major generations of the technology. Improvments in speed, efficiency and security have lead to the technology becoming a "fundamental utility [...], that everyone [...] expects to be available everywhere" [Sank 21, page 1]. WiFi connects billions of devices, like computers, smartphones, game consoles or IoT devices to the internet, generating a value of around 4.9 trillion USD worldwide [Sank 21, page 1].

WiFi operates at the physical (PHY) and the medium access control (MAC) layer of the OSI model, sending ethernet frames over radio waves in the 2.4 and 5 Ghz frequency band. The Mac layer is responsible for seting rules on tranmission and reception of data, connection management and power management. Data sent on the MAC layer then gets converted to radio waveforms by the PHY layer, which is also responsible for converting received data back to the MAC protocol. [Sank 21, page 4-7]

2.2. MAC layer

2.2.1. Architecture

Commonly WiFi networks have access points (AP), which are connected to the internet via ethernet, and stations (STA), which are provided with internet access over WiFi by the

AP. WiFi has a MAC layer, which is designed to be flexible and can be configured in the following topologies:

- **BSS**: Basic Service Set
- **IBSS**: ad hoc / peer-to-peer / Independent BSS
- **WDS**: Wireless Distribution System
- **MBSS**: Mesh BSS

BSS describes the most common and basic configuration, where multiple STAs are connected to one AP, which has wired connection to the internet. IBSS networks dont have any APs, instead the STAs are connected directly to each other. WDS can be used, when the reach of an AP needs to be extended, but there is no wired connection available. A secondary AP can be installed which receives backhaul connection from another AP. Mesh Networks are complex BSS networks with multiple APs, which the STAs can hop between. This type of network can improve the signal coverage and resilience and is often used in companies or public institutions with many STAs. [Sank 21, page 8-9]

2.2.2. MAC Frames

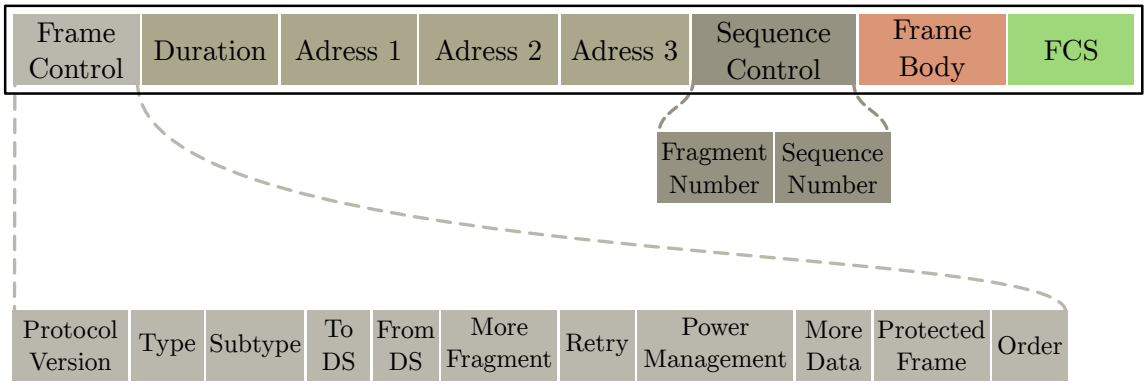


Figure 2.1.: MAC Frame of BSS WiFi Network

When data is sent over the MAC layer, it gets divided into chunks, which are called MAC Frames. MAC Frames have three components: the header, the body and the frame check sequence. The header contains information about the source and destination, the sequence and serveral control functions. In a BSS network it has the following functions:

- **Frame Control** contains 16 Bits which serve different control functions

  protocol version describes which version of the MAC protocol this frame uses

type describes the type of MAC frame is beeing used: there are data, management, control and acknowledgment frames

subtype describes the subtype of the frame, e.g. association request or beacon frame

from/to ds indicates the direction of travel

more fragment indicates more following fragments

retry indicates that the current frame is a retransmission

power management gives information about the power state of the STA

more data indicates more frames are to be transmitted

protected frame indicates whether a frame is encrypted

order set to 1 if the order of processing is important

- **duration** indicates the time of the transmission, therefore how long the medium is occupied
- **adress 1** contains the receiver address (RA), which can be the MAC address of the destination STA or the BSSID of the AP, depending on the direction, the frame travels
- **adress 2** contains the transmitter adress (TA), which can be the BSSID or the source address.
- **adress 3** contains either the source or destination address
- **sequence control**

  fragment number ensures the correct processing sequence of the fragments

  sequence number number of the frame

The Frame body then contains the payload, meaning the actual information to be sent. Every type of MAC frame is configured a different way to suit their specific function. "While data frames are meant to carry data, management frames are used for connection management and control frames [...] for triggering specific actions. Control frames have a smaller MAC header and no payload data [...]. Management frames have a structured payload composed of some fixed length fields and one or more variable length fields called information elements (IEs)" [Sank 21, page 12]

After the frame body comes the frame control sequence (FCS), which ensures the integrity of the frame. The FCS is a cyclic redundancy check (CRC), which is number calculated from

the frame header and body content. When the frame is compromised in traffic, the result of the CRC changes, therefore the error can be identified. [Geek 23] [Sank 21, page 10-12]

2.2.3. Medium Access

In LAN and WLAN networks all devices share the same medium, therefore only one device at a time can transmit information. Ethernet networks use full-duplex transceivers, meaning a device can transmit and receive data at the same time. This allows the transceiver to transmit and simlutaniously listen for collisions with other frames. Collision detection is quite effective, but can not be used in WiFi networks, because WiFi transceivers are only capable of half-duplex tranmission. WiFi instead uses a medium access control referred to as carrier sense multiple access collision avoidance (CSMA-CA).

When using this technique, the medium gets blocked during transmission and is freed by an acknowledgment frame, sent by the receiving device. All STAs start off in the random backoff state, where they listen for traffic. After the random backoff counter has expired and no traffic is present, the STA will start transmission. CSMA-CA can be described by the way "humans converse in a group, where every individual would first listen to check if someone else is talking, wait for the person talking to finish, and then start talking after a brief random pause" [Sank 21, page 12]. [Oriy 17] [Sank 21, page 12-17]

2.2.4. Connection Establishment and Termination

Before a connection can be established, the STA first needs to discover the AP it wants to connect to. This can be done by either active or passive scanning of all available channels.

During active scanning, the STA transmitts a management frames, called probe requests on all channels to all SSIDs in the vicinity. The APs answer with a probe respose frame, which contains the AP's SSID, BSSID and its capabilities.

Passive scanning works by the STA listening for beacon frames. Beacon frames are periodically sent out by APs and also contain information on SSID, BSSID and capability. [Sank 21, page 18]

Once the desired network is discovered the STA decides which AP to connect to, based on parameters, like the receive signal strength indicator (RSSI) or the APs capacity, performance and security. The STA then sends an authentication request, which the AP gives an authentication response to. If the credentials of the STA are valid and the authentication is successful, the STA proceeds to send an association request. The association response, sent by the AP, then indicates, if the connection was established successfully.

To terminate a connection, either the STA or the AP can send a disassociation or deauthentication frame, "containing a reason code that specifies the reason for termination" [Sank 21, page 20]. [Sank 21, page 19]

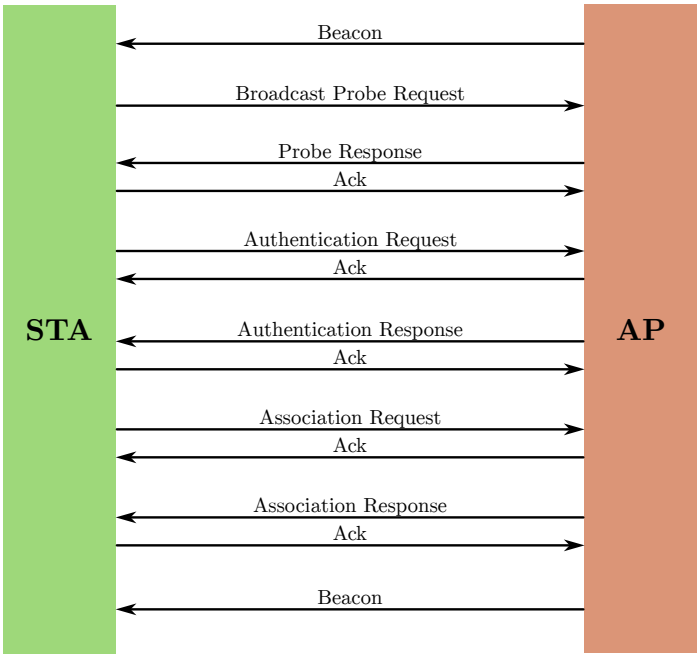


Figure 2.2.: Connection Establishment

2.3. PHY layer

The WiFi PHY layer is based on radio frequency (RF), therefore WiFi transceivers contain a transmitter and receiver circuit, as well as an antenna. When sending data, MAC frames are encoded and modulated onto a RF waveform which is then transmitted via the antenna. On arrival of data, the receiver circuit decodes the data, extracts the MAC frame, and provides it to the MAC layer. The first generations of WiFi used to transmit in the 2.4 Ghz band, but due to it beeing used by many other devices, such as cordless phones, computer peripherals, bluetooth devices or microwave ovens, WiFi 4 started using the 5 Ghz band, in order to improve connectivity. [Sank 21, page 5,34,35] With every generation of WiFi, the PHY layer components are modified to improve performance and efficiency, but due to the PHY layer not beeing majorly significant for cybersecurity, it will not be described further in this thesis.

2.4. Security

Through the widespread adoption of WiFi technology, for example in smart homes, infrastructure, public spaces, industry and goverments, the security of WiFi networks plays an ever increasing role. Therefore principles of cybersecurity need to be applied, in order to guarantee that networks are safe, reliable and performant. Cybersecurity aims to protect devices and data by achieving the CIA triad, which is an acronym for Confidentiality, Integrity, and Availability [Oriy 17] . Because of its wireless nature, "[i]t is difficult to confine the data-carrying radio waves to be within the physical security perimeter of [an] organizsation" [Sank 21, page 103], so anyone in the vicinity of a WiFi network could just intercept and manipulate network traffic. To ensure integrity and confidentiality, WiFi two main security mechanisms, which are encryption and authentication. [Sank 21, page 103]

Since this thesis is focused on private networks, the following summary of WiFi security mechanisms will not include enterprise authentication protocols.

2.4.1. Encryption

Encryption is a cryptographic [1] method, which takes the readable data, called plaintext, and makes it unreadable for anyone besides the recipient. The encrypted text is called ciphertext and can be made readable again, by decrypting it. To en- and decrypt a message, the sender and recipient have knowledge of a secret, often a passphrase, which controls en-/decryption and is called key. [Watj 18, page 1]

The first "secure" WiFi standard, called wired equivalent privacy (WEP) uses a shared key, with a RC4 stream cipher algorithm for message encryption and a 32-Bit CRC as a integrity check (IC). Due to serveral vulnerabilities of WEP, including the RC4 algorithm, a lack of authentication and no key management, the second standard, wireless protected access (WPA), was developed.

WPA still uses the RC4 cipher for compatibility reasons, but the encryption mechnism was updated, now called Temporal Key Integrity Protocol (TKIP), it uses a pairwise master key, which is generated for each STA after it has authenticated successfully. TKIP also dynamically changes the encryption key for each packet, which in combination with a longer 128-Bit preshared key (PSK) fixed some of the weaknesses of WEP. [Oriy 17]

WPA2 could be considered as the first proper WiFi security standard, since it replaced the weak RC4 algorithm with AES-CCMP[2] encryption. AES-CCMP encrypts data blockwise

[1] cryptography = science of secretive writing
[2] Advanced Encryption Security - Counter mode CBC MAC Protocol

and uses a compbination of substitution and transposition algorithms to acheive high security. WPA2 security also added protected management frames, so not only data frames, but also management frames are encrypted.

The newest WiFi security standard is WPA3, which aims to fix vulnerabilities of WPA2 and future proof the technology. Most changes for WPA3 have been made in authentication mechanisms, WPA3 added the optional support of AES-GCMP[3] encryption, which is more performant and even safer than AES-CCMP. [Sank 21, page 103-117]

2.4.2. Authentication

Authentication is the act of a user "performing some sort of action which proves that the claim a user is making about who they are is actually valid" [Oriy 17]. This is oftentimes done through the use of a password, in the case of WiFi networks, the knowledge of the password proves, that a user is allowed to access the network.

WEP authentication is only one-sided, where STA has to encrypt a package with the correct key, in order to complete association with the AP. The same key is used for encryption, making the whole system vulnerable, when ether the authentication or the encryption is exploited.

WPA and WPA2 networks use an authentication mechanism, that is two sided, where STA and AP exchange identity and capability. After entering the correct passphrase a password-based key derivation function then generates the PSK for encryption.

WPA3 replaces the PSK by simultaneous authentication of equals (SAE) [Oriy 17], [Sank 21, page ??]

Table 2.1.: Evolution of Wi-Fi Security [Sank 21, page 123]

	WEP	WPA	WPA2	WPA3
Certification start	-	2003	2004	2018
Mandatory since	-	2006	2006	2019
Information element	None	WPA IE	RSN IE	RSN IE
Encryption	RC4	TKIP	TKIP, AES-CCMP	AES-CCMP, AES-GCMP
Authentication	None	PSK, 802.1X	PSK, 802.1X	SAE, 802.1X
Open Wi-Fi security	None	None	None	OWE

[3] AES - Galois Counter Mode Protocol

Chapter 3.

Architecture

"IoT-Geräte sind häufig über ein Web-Interface zugänglich. Daher habe viele der Probleme bie IoT-Geräten mit unsichere Webanwendungen zu tun." [Hell 23, page 174]

Due to my limited experience in software development, the project does not focus on performance or software quality. Therefore the software is written to implement the wanted functionality and should be seen as experimental, since it is neither optimised for performance, nor have proper error handling mechanisms been implemented.

3.1. Hardware

The Project is realized using a RaspberryPI as the central device, connected to which are an external network card, an ESP32 microcontroller, a display and a rotary encoder for user input.

As the main computer the RaspberryPI 4B with 8GB of RAM was chosen due to its ease of use, linux based operating system and widely available documentation.

To be able to also test on WPA3 networks, an external network card is needed, due to the internal network card of the RPI not supporting the ciphers used by WPA3. The Fenvi AX1800 USB WiFi card was chosen for its compatibility with the linux kernel, according to the USB-Wifi github by user morrownr [NR 25]. Another factor for this card was the low pricepoint as well as its smaller formfactor, which lets it fit inside case for the device.

Selecting from the large offer of ESP32 MCUs, again size was taken into account and the ESP32-C6 Mini by Waveshare was chosen for its small size and good feature set. It connects to the RPI's GPIO pins using simple jumper cables, which supplies the ESP32 with power and establishes a UART connection.

Another device connected to the RPI's GPIO header is the rotary encoder. On its five pins, it receives 3.3V supply voltage and outputs a gray code, indicating the direction of rotation, on two of its outputs. The fith pin is used for the pushbutton inside the rotary encoder.

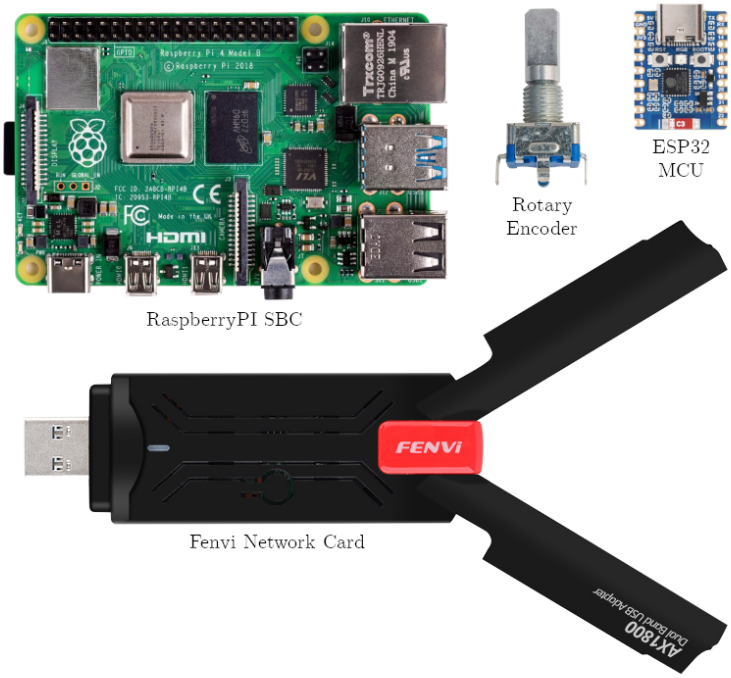


Figure 3.1.: Hardware Devices

3.2. Software

The software of the project is mostly modular and it consists of a central program, which runs the user interface and then executes various shell scripts, with which the functions are implemented. When executing the setup script, the main program, which is written in C++, gets compiled and the executable gets called by the start.sh script. The program then displays the user interface, which is a simple interface, printed into the console. Each page of the UI is a "Menu" object, each menu has a title, options of the type "MenuOptions", a variable to track the currently selected option, a method to add options to a menu and methods for navigation, input capture and displaying. MenuOption objects have a name, as well as an action of the std::function type, which can be executed by calling the execute() method. In the main of the program, the constructors for menues and the addOption() methods are beeing used, to create the different menues and their submenues. The program starts by calling the navigate() function of the main menu, which then displays all available submenues and menu options. All submenues work in the same way as the main menu, getting their respective navigate() called upon selection. During navigation the UI peridiodically prints the available options, using unicode to highlight the currently selected option as well as to give contrast to the title of the menu.

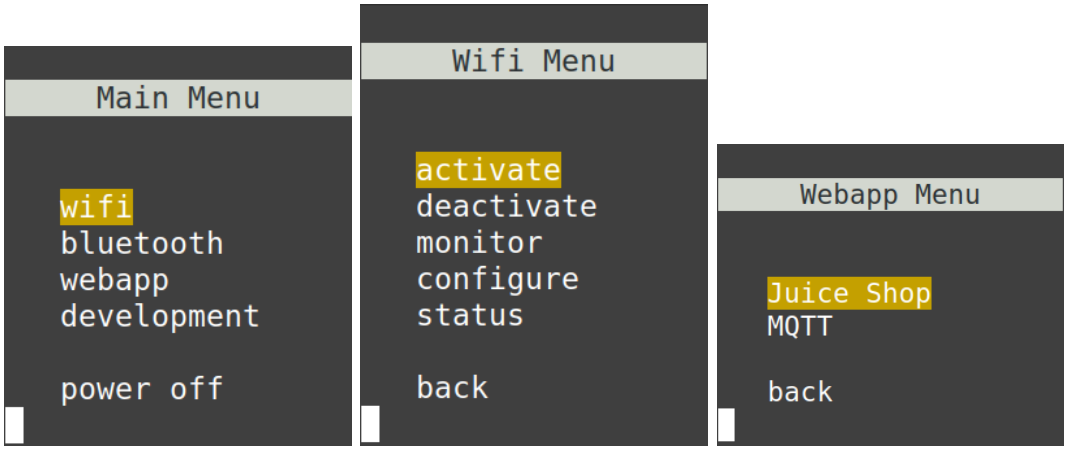


Figure 3.2.: Device Menues

For input the device uses a rotary encoder with a pushbutton, turning the encoder moves the selection up or down in the menues, the pushbutton executes the selected option. The encoder is connected to the RaspberryPIs GPIO interface and the input signals are read and processed by the encoder program. The start.sh script starts both programs and pipes the stdout of encoder into the stdin of main. When an action of the encoder is detected, the according command gets sent to main, the commands beeing the unicodes for the up/down arrows and the enter key on a keyboard. For example, the User turns the the encoder one click clockwise, therefore main receives one downwards arrow symbol and the selections moves to the next option below. Because no special commands are utilized for navigation, the program can be easily tested on any computer by navigating with the keyboard.

When an action is executed, main calls the according shell script with the system() function. The wifiActivate.sh script is used to open wifi hotspots, where the user can choose the type of Wifi standard beeing used. It receives an input command and then establishes a WEP, WPA, WPA2 or WPA3 network.

Listing 3.1: extract of wifiActivate.sh

```
#!/bin/bash

scriptdir="$(dirname␣"$0")"

read -n 4 connection

if [ "$connection" == "WEP" ]; then
   exists=$(nmcli connection show | sed -n '/Wifi-WEP␣/p')
   if [ -z "$exists" ]; then
      sudo nmcli device wifi hotspot con-name "Wifi-WEP" ssid "WEPtest" password "
          TestSetup123" ifname wlan1
      sudo nmcli connection down Wifi-WEP
      sudo nmcli connection modify Wifi-WEP 802-11-wireless-security.key-mgmt none
      # sudo nmcli connection modify Wifi-WEP 802-11-wireless-security.group wep104
      sudo nmcli connection modify Wifi-WEP 802-11-wireless-security.wep-key0 "test1"
   fi
      sudo nmcli connection up Wifi-WEP
```

With the wifiReset.sh script, the user can deactivate the active network. Once a network has been cracked, the wifiNewPassword.sh script can be used to give a new random password to the selected network. The script the pulls a random password out of a password list, which contains popular passwords that have been filtered to work with the specification of each network standard. To display the current status of the network, meaning its security standard, SSID and password, the wifiStatus.sh script is used.

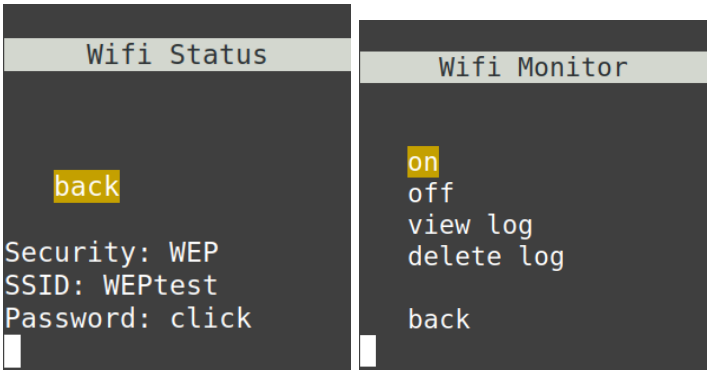


Figure 3.3.: WiFi Status and Monitor

After a WiFi hotspot has been activated, a command with the connection details is sent to the ESP32 over uart. This functionality is implemented by the uartSendNI.sh script, which gets called from wifiActivate.sh. Receiving the network information, the ESP32 will then connect to the provided network and act as a communication partner for the RPI.

Selecting the "monitor" option, the user is presented with the options of de-/activating the network monitoring as well as viewing or deleting the monitoring log. When activating monitoring, wifiMonitor.sh uses the `iw event -T` command to log devices connecting and disconnecting from the network.

Listing 3.2: extract of wifiActivate.sh

```
#!/usr/bin/bash

read -n 4 connection

UART_DEVICE="/dev/ttyS0"
UART_SPEED=115200

stty -F $UART_DEVICE speed $UART_SPEED cs8 -cstopb -parenb

ssid=$($(dirname "$0")/wifiStatus.sh | sed -n "s/SSID:␣//p")
password=$($(dirname "$0")/wifiStatus.sh | sed -n "s/Password:␣//p")

echo "reset" > "$UART_DEVICE"
sleep 1

DATA="wifi␣$connection␣$ssid␣$password"
```

```
echo "$DATA" > "$UART_DEVICE"
echo "Data sent: $DATA"
```

Inside the webapp menu the JuiceShop and MQTT communication can be found. When activating the JuiceShop, the juiceShop.sh script will start up the server for the OWASP JuiceShop project and open a hotspot, which can be used to access the simulated webstore.

When a WiFi network has already been activated from the WiFi menu, a simulated MQTT communication can be started from the MQTT menu. Starting a conversation will again send a command via uart to the ESP32, prompting it to connect to the MQTT broker, running on the RPI. Following up uart.sh starts the uart program, which starts an infinite MQTT conversation with the ESP32.

The development menu contains options used during development, these can be adjusted on the fly in the development.sh script.

When selecting the shutdown option, all running processes are stopped and the shutdown.sh script calls `shutdown now` to turn off the device.

Chapter 4.

Worklog

4.1.

The idea for the project started off as a collection of SD-Cards, containing operating systems with different functions. To use a function, the user should insert the wanted SD-Card into the RPI and on startup a script would run and configure the OS for a specific scenario.

With the usability of this system beeing questionable, I instead opted to develop a user interface, which would execute the wanted shellscripts during runtime. A simple interface, printed into the shell terminal seemed to be a simple way of implementing the UI and I decided to use a dial and pushbutton combination for intuitive navigation. The program should be written in C or C++ due to my familiarity with both languages.

I started off to find a way to display the menues in the terminal and landed on a simple loop, that peridiodically clears the screen and prints the options of a menu. Subsequently a software architecture needed to be designed, which proved challenging in C++, because nearly every menu is different and needs to execute different actions. The menues need to be able to display options, which need to be able to display another menu, trigger a shell script, or return to the previous menu. Since C++ does not have decorators, it is hard to model the menues with a uniform class, therefore it was decided to use a architecture, which models menues as classes, containing options and the options a class with a title and a std::function as the action.

For network managment, I decided to use the Network Manager Command Line Interface (nmcli) tool, which is easy to use and offers a sizable set of configuration options and utilities. Starting to write the shell scripts for WiFi activation, it was unclear, how to configure a connection in order to get it to use the different WiFi standards. Ater researching the different security mechanisms and reading the nmcli documentation the solution seems to be to manually configure the encryption algorithm, key management and information element for each connection.

Next up the script for changeing the WiFi passwords was developed, which was planned to use a password list and extract a random password from the list to use for the connection. Large password lists, like the rockyou list from kali linux proved to be impractical due to the very large size, as well as most of the passwords beeing unsuitable. WEP networks for example need a password, that is 5 or 13 characters long, therefore a password list with 100.000 entries was chosen as the data basis and filtered with a simple script, to get password lists, which match the requirements for each standard.

For the WiFi status function, the `nmcli dev wifi show-password ifname [device name]` can be used to display the type of network, its SSID and password. Unfortunately this command does not work for WEP networks, so I wrote a script, which extracts the SSID and password from */etc/NetworkManager/system-connections/[connection].nmconnection* and prints it in the same format as the nmcli utility.

With the WiFi monitor function I wanted to implement a simple logger, which lets the user retrace the events of an attack, such as devices dis-/connecting or changes to the network. `iw event -T` outputs the timestamp and MAC adress, when a device connects to or disconnects from the network.
`nmcli monitor` outputs any modifications made to the network status, but unfortunately does not output timestamps. I was also unsuccessful in printing timestamps with `awk`or `ts`, because ncmli monitor needs to be executed in the background to allow the UI to continue operating. Another problem, i have not been able to solve yet, is printing the output of both programs into a single logfile.

In order to later use the device with the build in screen and selector dial, the RPI needs to be configured to run and display the main program on startup. Running the program on startup is done by creating a systemd service, which can be done by adding a configuration file for that service to */etc/systemd/system* and running the `systemctl enable [service].service` command. Instead of a BIOS, the RPI has a configuration file (*/boot/firmware/config.txt*), which is read on startup to initialize the hardware and operating system. To this file I added the following lines:

dtoverlay=waveshare35a

framebuffer_width=200

framebuffer_height=120

The first line is for setting up an overlay to initialize the display, the other two lines are used to set the resolution. With the resolution set to the factory specification of 800x480, the UI is very small and not really suitable for the usecase. Since the OS only runs in the terminal without a graphic server, there is no easy options for scaling the image. Therefore I set a really low resolution of 200x120 to force the terminal to be displayed larger. Initially it was

planned to use the display in vertical orientation, but the attempts to rotate the terminal by configuration inside the config.txt or the kernel command line (*/boot/firmware/cmdline.txt*) were unsuccessful.

To use the encoder dial as a scrollwheel, its signals need to processed and translated to instructions for the main program. After soldering the encoder to a simple circuit of pull-up resistors and connecting it to the GPIO pins of the RPI, I enabled UART in the config.txt file and wrote the encoder.cpp program to process the inputs. Off the shelf rotary encoders have clicks similar to the scrolling wheel of a computer mouse and according to various online tutorials, the encoder switches one state on each click.

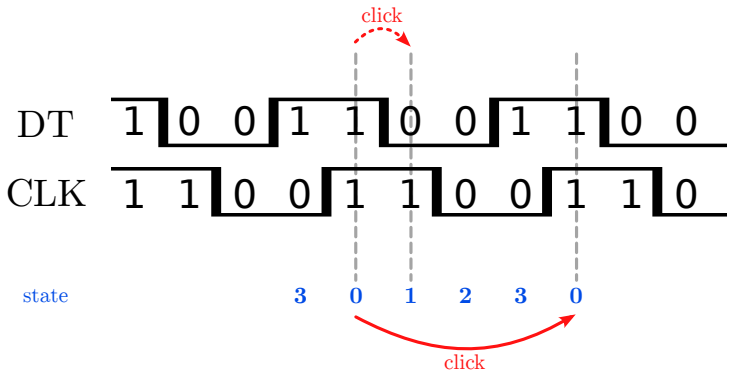


Figure 4.1.: Encoder Processing

Chapter 5.

Wifi Attacks

One of the earliest and most commonly used forms of attack used to target access control is a technique known as war driving. This technique, simply put, is to use a wireless enabled device along with specialized software used to detect or probe wireless networks that have come with in range of the wireless device. What makes war driving so popular and so effective is the fact that many computer users, both personal and business, have been deploying 802.11 wireless access points for years with little regard for security. The deployment of a wireless network that allows the user to roam has taken precedence over taking measures to secure their access points and devices against potential attack. In response to this deployment of wireless networks that are misconfigured or insecure, we have a category of attackers that engage in this practice known as war driving. Those who engage in this activity will build custom rigs built out of a combination of hardware and software, with the intent of cruising public venues to find both wireless access points and devices in some cases. These individuals may just be interested in locating and targeting a specific network for their own ends or, in some cases, they may even share this information to an online database or website where any visitor can search for these access points.

[Oriy 17]

5.1. Overview

WiFi attacks can take many different forms, depending on the type of attack and and which mechanism it tries to break. There are attacks aimed at breaking the Access Controls, Integrity Controls, Confidentiality or Availability.

Subsequently the following attacks will be explained and tested on the RPI in chapter 6:

- War Driving
- Rogue access point
- Evil Twin
- Cracking WEP
- MAC spoofing
- Deassociation

- Deauthentication
- Man-in-the-middle
- Cracking WPA
- KRACK
- Cracking WPA3

5.2. War Driving

War Driving describes the act of gathering information on WiFi networks in larger numbers in order to locate specific or weak targets. This is done by traversing areas of interest with data collection equipment, which collects information about every network coming into range. After capturing and mapping as many WiFi networks as possible, a more sophisticated attack can be launched on the captured networks or the data can be sold on the black market. [Oriy 17]

5.3. Rogue Access Point and Evil Twin

When using Rogue Access Points, attackers place extra APs onto a network, trying to stay undetected and get STAs to connect to the malicious AP. This can for example be done by connecting an AP to an open and unprotected LAN port inside a public or company building. Once a STA has connected to the rogue AP it acts as a MITM, allowing the attacker to manipulate or eavesdrop on the traffic.

Evil Twins have some similarity to Rogue Access Points, in that they also pretend to be a legitimate part of an existing network. Different to Rogue APs, Evil Twins do not connect to the legitimate network, but instead spoof [1] wireless networks, providing a stronger signal, to again get STAs to connect to it. Some attackers might also use a deauthentication flood to get STA disconnected from their genuine AP and autoconnect to the malicious one.

Because they only have one-way authentication, WEP and OWE networks are vulnerable to these attacks. [Sank 21, page 105,119]

5.4. Cracking WEP

After the release of the WEP protocol, security researchers quickly uncovered various flaws in the protocol, that lead to it beeing vulnerable to serveral exploits. Some of the most notable vulnerabilites are:

[1] replicate, fake, pretend

- One-way authentication
- Same key for authentication and encryption
-

Because WEP uses the same key for encrypting every frame, it can be reverse engineered by analyzing captured packets for reoccuring patterns.

Chapter 6.

Penetration Testing

6.1. Abc

Chapter 7.

Educational Use

7.1. abc

Chapter 8.

Outlook

Chapter 9.

Summary

Appendix A.

Supplemental Information

Anleitung zum Raspi-CyberSec-Lab

Jonas Schmitt

24. März 2025

1 Allgemeine Informationen

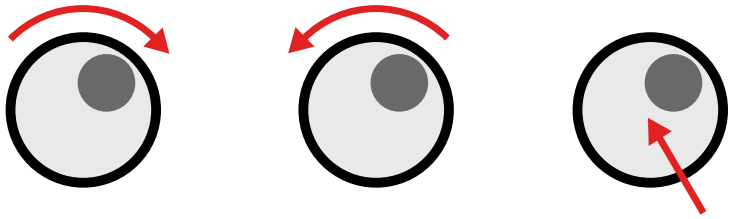
Das RaspberryPI Cybersecurity Lab ist eine Platform zum Testen verschiedener Cyberangriffe, das Projekt ist Teil meiner Bachelorarbeit in Elektro- und Informationstechnik an der Technischen Hochschule Nürnberg. Es hat verschiedene Funktionen um Umgebungen zu schaffen, welche für das Ausprobieren von Angriffen aus den Bereichen Wifi Netzwerke, Webseiten / -applikationen, MQTT Kommunikation und Bluetooth ausgelegt sind.

2 Benutzeranleitung

Zum Einschalten des Geräts, schließen Sie das Gerät mit dem zugehörigen Netzteil an den Strom an. Das Gerät fährt nun hoch, wobei der Bootvorgangn zu sehen ist (das RaspberryPI Logo wird angezeigt). Nach erfolgreichem Bootvorgang wird das Hauptmenü angezeigt.

2.1 Navigation

Zur Navigation wird das Drehrad, wie in folgender Abbildung beschrieben, genutzt:



nach unten **nach oben** **bestätigen**

Die farbige Hinterlegung zeigt an, welcher Menüpunkt aktuell ausgewählt ist. Mit dem Drücken des Knopfes wird die Auswahl bestätigt und das Untermenü geöffnet, bzw die Aktion ausgeführt.
Zum verlassen eines Menüs wird die Option "back"gewählt.

2.2 Menü Übersicht

Dem Nutzer steht der folgende Menübaum zu Verfügung:

- **wifi** - Optionen zu Wifi Netzwerken
  - **activate** - Aktivieren eines Hotspots
    Auswahl der Netzwerkart
  - **deactivate** - Abschalten des aktiven Hotspots
  - **status** - Anzeigen der aktuellen Netzwerkeinstellungen
  - **change password** - Setzen eines neuen Passworts
    Auswahl der zu änderenden Netzwerkart
  - **monitoring** - Einschalten und Anzeigen des Netzwerkmonitoring
    on - Einschalten
    off - Ausschalten
    show log - Log anzeigen
    delete log - Log löschen
- **bluetooth** - Optionen zu Bluetooth
- **webapp** - Optionen zu Webapplikationen
  - **Juice Shop**
    on - Server Einschalten
    off - Server Ausschalten
  - **MQTT**
    on - MQTT Konversation starten
    off - MQTT Konversation beenden
- **development** - Anpassbare Optionen für Entwickler
- **power off** - Ausschalten des Geräts

3 Installation

Das Projekt wurde mit folgender Hardware umgesetzt:

- RaspberryPI 4B mit 8GB RAM
- Fenvi AX1800 USB Netzwerkkarte
- Waveshare ESP32c6 Microcontroller
- Waveshare 4.3 zoll LCD
- Drehgeber mit Druckknopf

Als Betriebssystem des RaspberryPI dient RaspberryPI OS in der light Version (nur Kommandozeile). Zum Kompilieren und Flashen des ESP32 Codes wird das ESP-IDF benötigt, dieses kann in Visual Studio Code als Erweiterung installiert werden.

Zuerst muss das Github Repository des Projekts auf den RaspberryPI geklont werden, dies kann mit folgendem Kommando erfolgen:

```
git clone https://github.com/Der-Erzfeind/Raspi-CyberSec-Lab-Project.git
```

Es wird empfohlen das Repository in das standardmäßige Nutzerverzeichnis zu klonen. Folgende Aktionen müssen im Anschluss ausgeführt werden:

- Kompilieren von main.cpp, encoder.cpp und mqtt.cpp
- Alle Skripte ausführbar machen
- Installation und Konfiguration von Mosquitto
- Installation des JuiceShops (und node.js)
- Einrichten des systemd services zum Ausführen von start.sh beim Start des Geräts
- Kompilieren und Flashen des ESP32 Codes

List of Figures

2.1. MAC Frame of BSS WiFi Network 4
2.2. Connection Establishment 7

3.1. Hardware Devices 12
3.2. Device Menues 13
3.3. WiFi Status and Monitor 14

4.1. Encoder Processing 19

List of Tables

2.1. Evolution of Wi-Fi Security [Sank 21, page 123] 9

List of Listings

3.1. extract of wifiActivate.sh 13
3.2. extract of wifiActivate.sh 14

Bibliography

[Geek 23] GeeksforGeeks. “IEEE 802.11 mac Frame”. Mar 2023.

[Hell 23] R. Hellmann. *It-sicherheit: Methoden und Schutzmaßnahmen Für Sichere Cybersysteme.* De Gruyter Oldenbourg, 2023.

[NR 25] M. NR. “USB WiFi Adapters that are Supported with Linux In-Kernel Drivers”. 2025. Accessed: 2025-03-20.

[Oriy 17] S.-P. Oriyano. *Kali Linux Wireless Penetration Testing Cookbook: Identify and assess vulnerabilities present in your wireless network, Wi-Fi, and bluetooth enabled devices to improve your wireless security.* Packt Publishing, 2017.

[Sank 21] S. G. Sankaran and S. R. Gulasekaran. *Wi-Fi 6: Protocol and Network.* Artech House, 2021.

[Watj 18] D. Wätjen. *Kryptographie - Grundlagen, Algorithmen, Protokolle.* Vol. 3. Auflage, Springer Vieweg, 2018.

[Wiki 25] Wikipedia. “Wi-Fi”. Mar 2025.

Glossary

library A suite of reusable code inside of a programming language for software development. i, 1

shell Terminal of a Linux/Unix system for entering commands. i